

FastAPI

HANDWRITTEN ENGINEERING NOTES

APIs • Python • Backend • AI / ML / LLM

Compact, continuous, visual reference for revision + practical learning

1. API — CORE IDEA

API = Application Programming Interface

Definition: A defined way for one piece of software to ask another piece of software to do something or provide some information.

KEY IDEA

Think of an API as a waiter between two systems. You give a request; the waiter communicates with the kitchen/system and brings back the response.

Restaurant analogy

You say: "I want one masala dosa." The waiter carries the request to the kitchen and brings the result. You do not need to know how the kitchen works, where ingredients are stored, how the dosa is prepared, which cook prepared it, or how the kitchen communicates internally.

Weather example

Your phone → Weather API → Weather server/database → 28°C → Your phone

The API is the middleman that allows applications to communicate.

Google Maps / food delivery

A food-delivery app can ask a mapping service for Restaurant → Customer distance instead of building its own complete mapping system. Example response: Distance = 7.2 km.

2. WHY APIs? + WHAT IS FASTAPI?

Different applications/systems need a controlled and standardized way to communicate.

Mobile App → API → Order Service → API → Payment Service → API → Payment Gateway

Large systems may contain mobile apps, websites, payment, inventory, order, delivery, recommendation and AI systems. APIs provide the standardized doors between them.

API is NOT a programming language

API is a communication interface / convention, not Python, Java, JavaScript, FastAPI, C++, or SQL.

Possible technologies: Python → FastAPI / Flask / Django; Java → Spring Boot; JavaScript → Express.js; C# → ASP.NET; Go → Gin.

FastAPI

DEFINITION

FastAPI is a Python framework for building APIs and web backends. Think: Python + FastAPI → API / Backend.

3. PYTHON → API

Python without FastAPI

```
def add_numbers(a, b):  
    return a + b  
  
result = add_numbers(10, 20)  
print(result)
```

Output:
30

This function is directly usable by Python code that can access the program. A mobile app cannot simply call `add_numbers(10, 20)` over the network.

Expose the function with FastAPI

```
from fastapi import FastAPI  
  
app = FastAPI()  
  
@app.get("/add")  
def add_numbers(a: int, b: int):  
    return {"result": a + b}
```

Another application can request GET `/add?a=10&b=20`. FastAPI extracts the values, calls `add_numbers(10, 20)`, and returns `{"result": 30}`.

MENTAL MODEL

FastAPI is the bridge that exposes Python functionality through a network-accessible API endpoint.

4. ENDPOINTS + HTTP METHODS

Endpoint / route: a specific address through which another application can interact with your system.

YOUR API

- |— /users → Users
- |— /orders → Orders
- |— /add → Addition

Examples: GET /users, GET /users/123, POST /users, DELETE /users/123, GET /products, POST /orders.

GET / POST / PUT / DELETE

GET = retrieve information → GET /products → “Give me the products.”

POST = create something → POST /users → “Create a new user.”

PUT = update something → PUT /users/123 → “Update user 123.”

DELETE = delete something → DELETE /users/123 → “Delete user 123.”

SHOPPING ANALOGY

GET = Show me • POST = Create this • PUT = Change this • DELETE = Remove this.

Example URL: https://example.com/users → server = https://example.com; endpoint = /users.

5. JSON + FIRST COMPLETE API

What does an API return?

Most modern APIs commonly return JSON, a structured way to represent information.

```
{"name": "Likhith", "age": 30, "city": "Bengaluru"}
```

Think of JSON as a neatly organized digital form instead of an unstructured string such as “Likhith 30 Bengaluru”.

Complete FastAPI example

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "Hello from FastAPI!"}

@app.get("/add")
def add(a: int, b: int):
    return {
        "a": a,
        "b": b,
        "result": a + b
    }
```

Endpoint 1: GET / → {"message": "Hello from FastAPI!"}

Endpoint 2: GET /add?a=10&b=20 → {"a": 10, "b": 20, "result": 30}.

6. REQUEST → FUNCTION → RESPONSE

Client → HTTP Request → FastAPI Endpoint → Python Function → JSON Response → Client

Example: GET /add?a=10&b=20 → FastAPI receives it → sees @app.get("/add") → extracts a = 10, b = 20 → executes add(10, 20) → gets 30 → converts to JSON → sends it back.

Understanding the code

```
from fastapi import FastAPI
```

Import FastAPI.

```
app = FastAPI()
```

Create the FastAPI application — “Create my web/API server.”

```
@app.get("/add")
```

When a GET request arrives at /add, run the function below it.

```
def add_numbers(a: int, b: int):  
    return {"result": a + b}
```

The function contains the Python logic and returns the answer.

7. LOCALHOST + PORT 8000

127.0.0.1

Means “this computer”. Therefore 127.0.0.1:8000 = your computer + port 8000. localhost:8000 also refers to your own computer.

Port analogy

Computer = apartment building

Door 80 • Door 443 • Door 3000 • Door 5000 • Door 8000

These are ports. The FastAPI example listens on 8000. localhost:8000 means “talk to the application listening on port 8000 on this computer.”

Installation

Check Python:

```
python --version
# or
python3 --version
```

Expected: Python 3.x.x.

Create a project:

```
fastapi-learning/
└─ main.py
```

Create an isolated environment:

```
python -m venv .venv
```


8. ACTIVATE + INSTALL + RUN

Virtual environment = isolated Python workspace for the project. Separate projects can have separate packages and tools.

Activate

Windows:

```
.venv\Scripts\activate
```

macOS / Linux:

```
source .venv/bin/activate
```

Terminal usually shows (.venv).

Install FastAPI

```
pip install "fastapi[standard]"
```

This installs FastAPI plus standard tooling needed for the development server.

Create main.py

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "Hello FastAPI!"}
```

Start

```
fastapi dev main.py
```

You should see something similar to Uvicorn running on `http://127.0.0.1:8000`. Open `http://127.0.0.1:8000` → `{"message": "Hello FastAPI!"}`.

MILESTONE

You have created your first working API.

9. AUTOMATIC DOCS + FEATURES

Interactive documentation

Open <http://127.0.0.1:8000/docs>. FastAPI automatically creates an interactive API documentation page. You can click an endpoint and test it.

What FastAPI provides

1. API routing — `/users`, `/products`, `/orders`, `/login`, `/predict`, `/chat`.
2. Input validation — if an API expects `age: int` but receives `age = "hello"`, FastAPI can detect the invalid input.
3. Automatic documentation — generated API documentation.
4. JSON handling — straightforward input/output of JSON.
5. Type hints — `name: str`, `age: int`, `price: float` help FastAPI understand expected data.
6. High performance — designed for high-performance web APIs and uses the ASGI ecosystem.

KEY OBSERVATION

FastAPI is not merely a way to write URLs; it gives you a structured way to expose Python logic as a web service.

10. FASTAPI + AI / ML

A Python ML model can be exposed through an endpoint so a website, mobile app, or another service can use it.

```
def predict_house_price(area, bedrooms):  
    ...
```

Website → FastAPI → AI Model

Mobile App → FastAPI → AI Model

Another Service → FastAPI → AI Model

```
@app.post("/predict")  
def predict(area: float, bedrooms: int):  
    price = model.predict(area, bedrooms)  
    return {  
        "predicted_price": price  
    }
```

Other applications call POST /predict and can receive {"predicted_price": 8500000}.

WHY IT MATTERS

Request → FastAPI → Python model → JSON response is a fundamental pattern in production AI applications.

11. FASTAPI + LLM

Suppose an AI chatbot is implemented in Python:

```
def ask_ai(question):  
    answer = ...  
    return answer
```

A website cannot directly call that Python function over the network, so expose it as POST /chat.

User → Web Application → FastAPI → Python AI Logic → LLM → FastAPI → Web Application → User

```
from fastapi import FastAPI  
  
app = FastAPI()  
  
@app.post("/chat")  
def chat(question: str):  
    answer = ask_ai(question)  
    return {  
        "question": question,  
        "answer": answer  
    }
```

AI PATTERN

FastAPI acts as the communication layer between the client and your AI/LLM logic.

12. FASTAPI + DATABASE

FastAPI can receive a request, validate it, query a database, retrieve data, convert the result to JSON, and return it.

FastAPI → Users / Orders / Products → Database

Example request: GET /users/123.

Typical steps:

1. Receive the request.
2. Validate the user ID.
3. Query the database.
4. Retrieve the user.
5. Convert the result to JSON.
6. Return it.

Realistic user API

```
from fastapi import FastAPI

app = FastAPI()

users = [
    {"id": 1, "name": "Rahul", "age": 25},
    {"id": 2, "name": "Priya", "age": 28}
]

@app.get("/users")
def get_users():
    return users

@app.get("/users/{user_id}")
def get_user(user_id: int):
    for user in users:
        if user["id"] == user_id:
            return user
    return {"message": "User not found"}
```

13. PATH + QUERY PARAMETERS

Path parameters

```
@app.get("/users/{user_id}")
```

The URL contains a variable. Examples: `/users/1`, `/users/2`, `/users/100`. For `/users/2` → `user_id = 2` → Python executes `get_user(2)`.

Query parameters

```
/products?category=shoes&limit=10
```

Here `category = shoes` and `limit = 10`.

```
@app.get("/products")
def get_products(category: str, limit: int = 10):
    return {
        "category": category,
        "limit": limit
    }
```

Request: `/products?category=shoes&limit=5` → Response: `{"category": "shoes", "limit": 5}`.

14. REQUEST BODY + PYDANTIC

For POST requests, we often send a JSON body.

```
{"name": "Rahul", "age": 25}
```

FastAPI can define exactly what it expects. This is where Pydantic becomes important.

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class User(BaseModel):
    name: str
    age: int

@app.post("/users")
def create_user(user: User):
    return {
        "message": "User created",
        "user": user
    }
```

PYDANTIC

Defines the expected structure and validates incoming data before your Python code processes it. Think of it as a checking officer.

If expected input is `age: int` but someone sends `age = "banana"`, the input does not match the expected type/structure.

15. BIGGER PICTURE — PRODUCTION AI BACKEND

INTERNET



FastAPI Server



Users / Orders / Products



Database



AI Model + Cache



LLM

A FastAPI server can sit between clients and Python business/AI logic, databases, caches, and LLMs.

Why learn FastAPI?

It moves you from “I have a Python program” to “I have a service that other applications can use.”

Beginner script: Python script → reads file → processes data → prints result.

Backend: Client → FastAPI → Python → Database.

AI application: Frontend → FastAPI → Agent → Tools → Database / APIs → LLM.

KEY OBSERVATION

This is much closer to how production AI applications are built.

16. LEARNING ROADMAP

LEVEL 1 — Python fundamentals

Variables • data types • if/else • loops • functions • lists • dictionaries • classes • exceptions • modules • virtual environments • pip • type hints.

LEVEL 2 — Web/API fundamentals

Client • Server • HTTP • Request • Response • URL • Endpoint • GET • POST • PUT • DELETE • Status codes • JSON • Headers • Query parameters • Path parameters • Request body.

LEVEL 3 — FastAPI fundamentals

FastAPI() • @app.get() • @app.post() • Path parameters • Query parameters • Request bodies • Pydantic • Response models • HTTP status codes • Error handling • Swagger/OpenAPI.

LEVEL 4 — Real FastAPI

Project structure • Routers • Dependencies • Authentication • Middleware • Database • SQLAlchemy • Async programming • Testing • Environment variables • Logging • Docker • Deployment.

LEVEL 5 — AI + FastAPI

FastAPI + LLMs + RAG + Vector databases + Agents + Tool calling + Streaming + Authentication + Observability.

FOCUS

Learn each level in sequence. Do not try to learn all of FastAPI at once.

17. 7-DAY FIRST PROJECT PLAN

AI Assistant API

AI Assistant API

├─ /health

├─ /chat

└─ /users

↓

LLM

DAY 1: FastAPI installation → main.py → / → /hello → /add → run server → /docs.

DAY 2: GET → POST → path parameters → query parameters → JSON → Pydantic.

DAY 3: CRUD API → Users → Products → Orders.

DAY 4: Database → SQLite → SQLAlchemy.

DAY 5: Authentication → JWT → password hashing.

DAY 6: Connect an LLM → /chat.

DAY 7: Build a small AI API.

18. MASTER REVISION / FORMULA SHEET

CORE DEFINITION

API = communication interface between software systems.

FASTAPI

FastAPI = Python framework for building APIs and web backends.

CORE RELATION

Python + FastAPI → API / Backend

REQUEST PIPELINE

Client → HTTP Request → FastAPI Endpoint → Python Logic → Response → Client

AI PIPELINE

Frontend → FastAPI → Agent / Python AI Logic → Tools / Database / APIs → LLM → FastAPI → Frontend

Endpoint: a specific address such as /users or /chat.

GET: read • POST: create • PUT: update • DELETE: remove.

Path parameter: /users/{user_id}. Query parameter: /products?category=shoes&limit=10.

Request body: JSON data sent with requests such as POST.

Pydantic: defines and validates expected input structure/types.

localhost / 127.0.0.1: this computer. Port 8000: example FastAPI server port. /docs: interactive API documentation.

INTERVIEW LINE

FastAPI lets Python applications expose well-defined HTTP endpoints with validation, JSON handling and automatic API documentation, making it especially useful for modern backend and AI/GenAI services.

ONE PICTURE TO REMEMBER

API = "communication door"

Client → FastAPI Server → Python Logic

├→ Database
├→ AI
└→ LLM

API = communication interface. FastAPI = Python framework that builds the interface. Python = actual application logic.